



Lista 6 — Projetos e Deploy

Aulas 51 a 60

Desenvolvimento Web

Exercícios sobre projetos completos, deploy, CI/CD, performance e esteira de desenvolvimento. Cenários de entrega contínua e operação de aplicações web.

Exercício 1

Cenário: Projeto final — e-commerce completo com front-end em React + TypeScript.

1. Planeje a arquitetura: página inicial, catálogo com filtros, carrinho, checkout, área do cliente. Desenhe o diagrama de componentes do React.
2. Implemente gerenciamento de estado global (Context ou Zustand) para o carrinho compartilhado entre páginas.
3. Crie testes E2E com Playwright ou Cypress: fluxo completo de compra (selecionar produto, adicionar ao carrinho, checkout, pagamento).
4. Documente o projeto com README: descrição, stack, instruções de instalação, links de deploy.

Exercício 2

Cenário: CI/CD — deploy automático na Vercel/Netlify a cada push na main.

1. Conecte o repositório GitHub à Vercel ou Netlify: deploy contínuo com preview de PRs.
2. Configure variáveis de ambiente (VITE_API_URL, VITE_API_KEY) nos provedores de deploy.
3. Adicione GitHub Actions: workflow que roda lint + testes + build e só faz deploy se tudo passar.
4. Pesquise Docker para front-end: crie um Dockerfile multi-stage (build com node:20-alpine, serve com nginx:alpine). Qual a vantagem de servir com nginx em vez de dev server?

Exercício 3

Cenário: Otimizar performance do site que está lento (LCP > 4s).

1. Identifique as causas comuns: imagens não otimizadas, render-blocking resources, JS bundle grande, cache ausente.
2. Implemente lazy loading para imagens (`loading="lazy"`) e code splitting com `React.lazy` + `Suspense`.
3. Use Lighthouse (Chrome DevTools) para auditar a performance atual e após as otimizações. Compare as pontuações.
4. Pesquise Core Web Vitals: LCP (< 2.5s), FID (< 100ms), CLS (< 0.1). Como cada métrica é medida e quais técnicas melhoram cada uma?

Exercício 4

Cenário: Monitoramento e observabilidade — app em produção com milhares de usuários.

1. Integre um serviço de monitoramento de erros (Sentry) para capturar exceções no front-end.
2. Implemente analytics com Google Analytics 4 ou Plausible: eventos personalizados (`add_to_cart`, `purchase`, `signup`).
3. Crie logging estruturado no front-end: `console.info`, `console.warn`, `console.error` com contexto (componente, ação, dados relevantes).
4. Pesquise o uso de feature flags (LaunchDarkly ou Unleash) para fazer deploy de funcionalidades desligadas e ativar gradualmente (canary release).

Requisitos

Node.js 18+, Vercel/Netlify (conta gratuita). GitHub para Actions. Sentry (conta gratuita). Docker (opcional).